

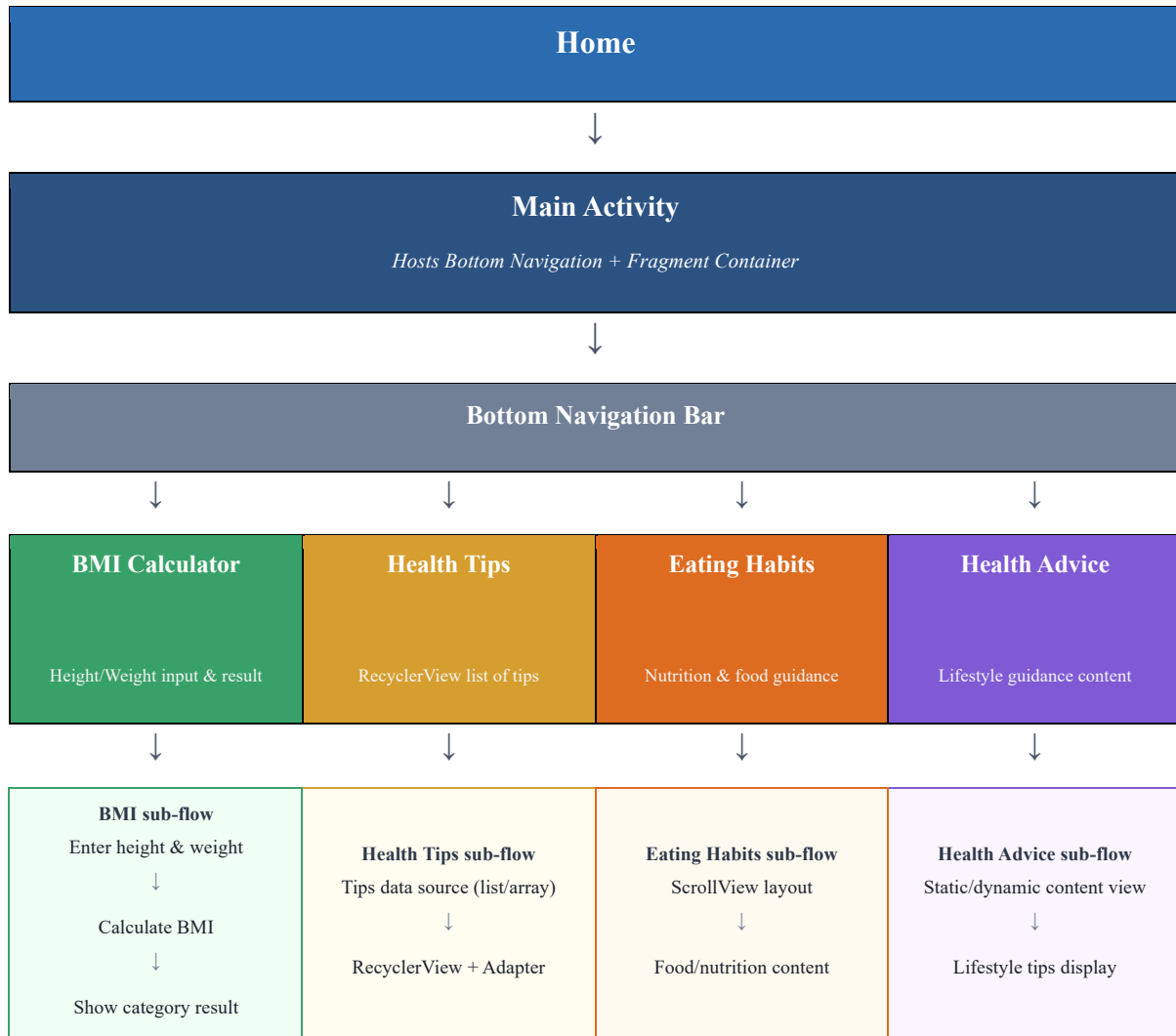
HEALTH & WELLNESS

Android Application - Assignment Submission
CINEC CAMPUS

Deshan Siriwardhana- B005-B/0009

1. Architecture Diagram

The application is built using a modern Jetpack Navigation Component with Single-Activity Architecture (MainActivity). Navigation between the primary screens is managed through a central BottomNavigationView linked to a NavHostFragment. This eliminates redundant navigation logic across activities, ensuring clean separation of concerns and maintainable lifecycle management.



2. UI Design

This section records all the main screens within the application. For each section below, insert a labeled screenshot representing the screen in the app.

2.1. Welcome & Home Screen (`HomeFragment` / `MainActivity`)

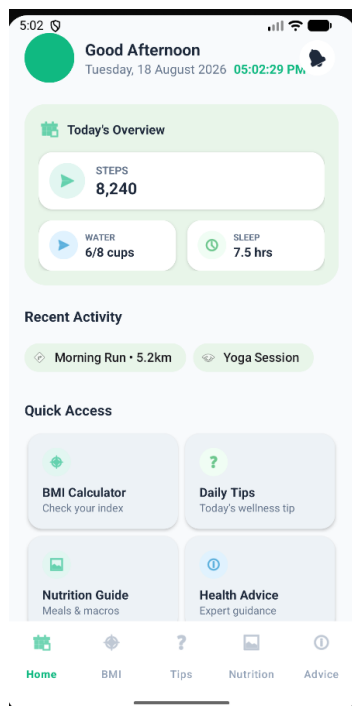
The primary landing screen when running the app. It displays user greeting, datetime, overview metrics, and quick action shortcuts.

Dynamic time-based greeting (Good Morning / Afternoon / Evening) and date header.

Today's Overview cards: Steps (8,240), Water (6/8 cups), and Sleep (7.5 hrs).

Recent Workout Activity Chips (Morning Run, Yoga Session).

Quick Access shortcuts to all main tools.



2.2.BMI Calculator (`BmiFragment`)

Allows the user to enter their measurements, perform unit conversions, and interpret their Body Mass Index.

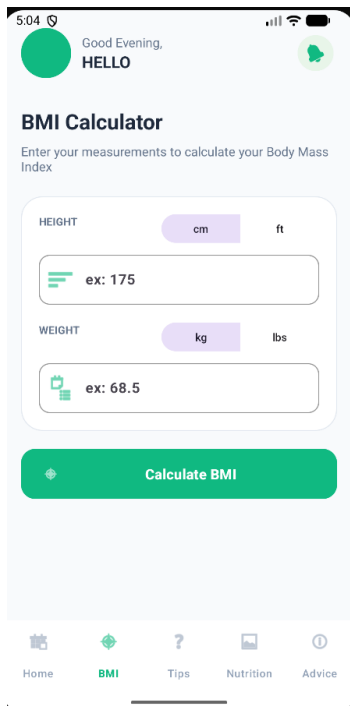
Input fields for Height and Weight.

Unit toggles for Height (`cm` / `ft`) and Weight (`kg` / `lbs`).

Calculate button triggering instant calculation.

Dynamic result card displaying numeric value, status category, and tailored health advice.

Custom `BmiGaugeView` rendering circular category gauge with animated needle.



2.3. Tips Screen ('TipsFragment')

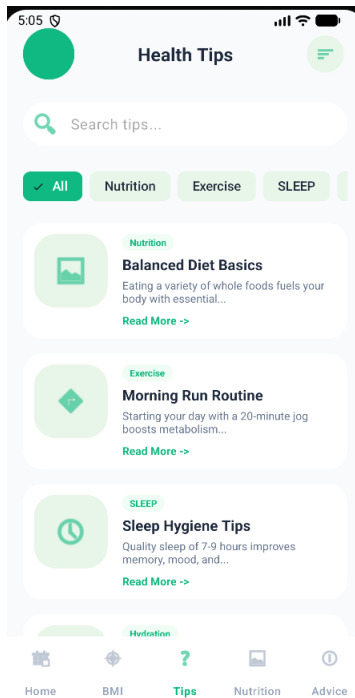
A scrollable collection of daily health and wellness advice.

Category Filter Chips ('All', 'Nutrition', 'Exercise', 'Sleep', 'Hydration', 'Mental Health').

Live search bar filtering tips dynamically as text is entered.

Custom 'TipsAdapter' binding data into a 'RecyclerView'.

Clickable cards opening detailed takeaway dialogs.



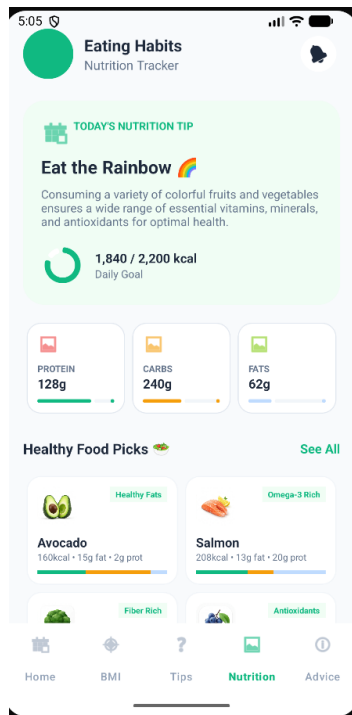
2.4. Eating Screen ('NutritionFragment')

Presents nutritional advice, calorie targets, macro tracking, and healthy food choices.

Calorie counter (1,840 / 2,200 kcal) and progress indicators for Protein (128g), Carbs (240g), and Fats (62g).

Nutritional guidance section ("Eat the Rainbow 🌈").

2-Column Grid of 3D illustrated food cards (Avocado, Salmon, Broccoli, Blueberries, Eggs, Chicken) with "Add to Meal Log" interaction.



2.5. Advice Screen (`AdviceFragment`)

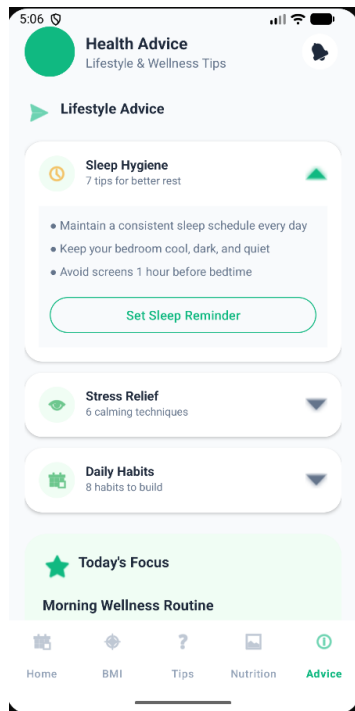
Provides lifestyle and wellness guidance beyond diet and exercise.

Expandable Sleep Hygiene card with an interactive "Set Sleep Reminder" button.

Stress Relief techniques (4-7-8 Breathing, Digital Detox, Journaling).

Daily Healthy Habits list (8 Habits to Build).

Interactive "Start Routine" guided morning session trigger.



3. Functionality Overview

This section describes in code the implementation of core logic and UI components of the app: BMI calculation flow, dynamic UI feedback, custom gauge rendering, and state management.

3.1.BMI Calculation Logic

The BMI calculation is handled in `BmiFragment.kt` and is executed when the user taps the Calculate button. The listener performs the following:

1. Reads raw text input from Height and Weight EditText fields.
2. Converts input to metric values if imperial toggles (`ft` or `lbs`) are checked.
3. Converts height from centimeters to meters (`height / 100f`).
4. Computes BMI using standard formula: $BMI = \frac{Weight(kg)}{(Height(m))^2}$.
5. Evaluates the resulting BMI float against an `when` condition block to classify into: Underweight, Normal, Overweight, or Obese.

```
kotlin
// Read and convert inputs
var height = binding.etHeight.text.toString().toFloat()
var weight = binding.etWeight.text.toString().toFloat()
```

```

if (binding.heightToggle.checkedButtonId == R.id.btn_ft) {
    height *= 30.48f // Convert ft to cm
}
if (binding.weightToggle.checkedButtonId == R.id.btn_lbs) {
    weight *= 0.453592f // Convert lbs to kg
}

height /= 100f // Convert cm to meters
val bmi = weight / (height * height)

// Category classification
val category: String
val color: Int
when {
    bmi < 18.5f -> {
        category = "Underweight"
        color = android.graphics.Color.BLUE
    }
    bmi < 25.0f -> {
        category = "Normal"
        color = resources.getColor(R.color.primary_green, null)
    }
    bmi < 30.0f -> {
        category = "Overweight"
        color = android.graphics.Color.parseColor("#FDE047")
    }
    else -> {
        category = "Obese"
        color = android.graphics.Color.RED
    }
}
}

```

3.2.Dynamic UI Updates

As soon as a category is defined, the UI automatically adapts to reflect the change:

The result card visibility toggles to View.VISIBLE.

tvBmiValue updates with the formatted float score (String.format("%.1f", bmi)).

tvBmiCategory updates its text and changes the text color to match the corresponding health category.

bmiGauge.setBmi(bmi) passes the calculated float value to update the gauge needle position..

3.3. Custom UI Rendering - `BmiGaugeView`

In order to avoid static images for visualization, the app uses a custom subclass of View (BmiGaugeView.kt) which overrides onDraw(Canvas canvas) to draw the gauge components manually:

canvas.drawArc() draws the 4 category arc segments (Underweight, Normal, Overweight, Obese) using custom Paint strokes.

Computes needle angle dynamically based on BMI value ($\theta = 180^0 + ratio * 180^0$).

canvas.drawLine() draws the animated indicator needle pointing directly to the user's score on the arc.

```
kotlin
override fun onDraw(canvas: Canvas) {
    super.onDraw(canvas)

    // Draw 4 color category arcs
    canvas.drawArc(rectF, 180f, 45f, false, underweightPaint)
    canvas.drawArc(rectF, 225f, 45f, false, normalPaint)
    canvas.drawArc(rectF, 270f, 45f, false, overweightPaint)
    canvas.drawArc(rectF, 315f, 45f, false, obesePaint)

    // Calculate needle angle and draw indicator line
    val angle = calculateNeedleAngle(currentBmi)
    val startX = width / 2f
    val startY = height / 2f + 40f
    val endX = startX + radius * cos(Math.toRadians(angle.toDouble())).toFloat()
    val endY = startY + radius * sin(Math.toRadians(angle.toDouble())).toFloat()

    canvas.drawLine(startX, startY, endX, endY, needlePaint)
}
```

3.4 State Management

The application favors clean, predictable, stateless logic for calculations:

User inputs (height and weight) are held in memory during calculation.

BMI results are calculated synchronously on the main thread and immediately reflected in the UI.

Filter chips and search queries in TipsFragment operate in-memory on the master list for instant response without database overhead.

Simple, traceable method chain: input→Parse→Calculate→Classify→Render.

4. Android Studio file & APK file link

https://drive.google.com/file/d/19xEwZbYdG_fe7nysbLl-3FGtBT9mGVIL/view?usp=sharing